

LoRaTrimmer: Optimal Energy Condensation with Chirp Trimming for LoRa Weak Signal Decoding

Jialuo Du
dujl20@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Yunhao Liu
yunhao@tsinghua.edu.cn
Tsinghua University
Beijing, China

Yidong Ren
renyidon@msu.edu
Michigan State University
Michigan, USA

Li Liu
liuli95@mail.tsinghua.edu.cn
Tsinghua University
Beijing, China

Zhichao Cao
caozc@msu.edu
Michigan State University
Michigan, USA

Abstract

LoRa has been widely used for the Internet of Things (IoT) due to its low power consumption and long communication range. The standard LoRa demodulation process condenses the energy of LoRa chirps to combat noise. However, there is an intrinsic frequency jump in real-life LoRa signals that standard demodulation neglects, reducing communication range in practice. We thoroughly study the frequency jump phenomenon and observe that it affects LoRa demodulation mainly in two folds: First, it makes each section of the signal shorter than the standard FFT perception range, introducing additional noise; Second, it induces a random phase jump that causes destructive addition of signal power. To mitigate the influence of frequency jump on LoRa demodulation, we propose LoRaTrimmer, a novel, fast, and noise-resilient LoRa decoding algorithm that optimally condenses LoRa signal power. LoRaTrimmer contains two innovative designs: First, we trim the perception range of FFT at the frequency jump, trimming off the additional noise; Second, we bypass the phase jump induced by frequency jump by probabilistic modeling and add up signal power constructively. Furthermore, we performed theoretical analysis to guarantee the performance of our method. Thorough experiments in various real-life environments show 1.70 to 2.49 dB SNR gain over the state-of-the-art and 3.44 to 3.79 dB SNR gain over FFT-based methods, translating to at most 1.67 times gain of coverage area. LoRaTrimmer is also robust under complex

noise patterns, and capable of real-time decoding, with the only overhead being a slight increase in computational cost (0.51 to 3.17 ms per packet, compared with 0.23 to 0.94 ms of baseline methods).

CCS Concepts

• **Networks** → **Network protocols**; **Network algorithms**.

Keywords

Internet-of-Things, Low-Power Wide Area Networks, LoRa

ACM Reference Format:

Jialuo Du, Yunhao Liu, Yidong Ren, Li Liu, and Zhichao Cao. 2024. LoRaTrimmer: Optimal Energy Condensation with Chirp Trimming for LoRa Weak Signal Decoding. In *The 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24)*, November 18–22, 2024, Washington D.C., DC, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3636534.3690681>

1 Introduction

LoRa [1] is widely used in Low-Power Wide-Area Networks (LPWANs), connecting Internet of Things (IoT) devices into various applications (e.g., smart industry, smart city, smart agriculture) [21, 24, 37]. The Chirp Spread Spectrum (CSS) mechanism enables LoRa devices to communicate at low bit rates at extremely low Signal-to-Noise Ratio (SNR), e.g., below -15 dB, achieving communication distances of up to tens of miles in ideal settings [20]. In LoRa, CSS expands each LoRa symbol over a long period of time (called “chirp”), accumulating enough signal transmit power in each symbol. However, during transmission, this signal will be contaminated by noise. The standard LoRa demodulation method uses a dechirp + FFT process to enable noise-resistant decoding. The dechirp process condenses the signal energy into a single frequency, and the FFT process evaluates signal power at all frequencies, identifying the signal from noise and thus decoding the signal.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ACM MobiCom '24, November 18–22, 2024, Washington D.C., DC, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0489-5/24/11

<https://doi.org/10.1145/3636534.3690681>

Table 1: Comparison between our method and recent studies on LoRa low-SNR communication.

	Approach	Improvement	Overhead
Charm [5]	Additional Spatial Diversity	1 to 3 dB	2 to 8 Gateways
Xcopy [35]	Additional Symbols	3 dB	$\geq 2\times$ Extra Power
LoRaPhy [36]	Energy Condensation	0.5 to 0.8 dB	$\geq 2\times$ Oversampling
Choir [7]	Superimposed Signals	N/A	10 to 30 Synced Transmitters
MALoRa [14]	Antenna Diversities	N/A	2 to 8 Synced Receivers
NeLoRa [17]	Pattern Recognition	1.8 to 2.4 dB	Heavy Computation: 7.19 to 32.24 ms/packet*
LoRaTrimmer	FFT trimming & Prob. Modelling	3.4 to 3.8 dB	Light Computation: 0.13 to 0.97 ms/packet*

* Computational time for decoding a 16-symbol LoRa packet on a PC with AMD Ryzen 9 5950X CPU and one Nvidia 2080 Ti GPU.

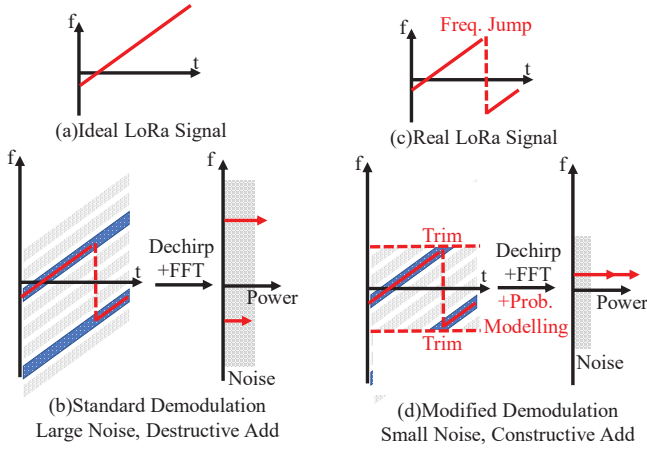


Figure 1: The frequency jump intrinsic in LoRa symbols induces additional noise and induces a phase jump that causes destructive add of power. LoRaTrimmer trims the FFT perception range and uses probabilistic modeling to improve LoRa decoding under low SNR.

However, the communication range of LoRa is often lower than expected in urban areas with plenty of obstruction and interference, which calls for improvements on the standard LoRa demodulation method [6, 17, 28]. Blockage, attenuations, and interferences in urban areas will severely reduce the SNR of LoRa signals, requiring LoRa transmitters to use more transmitting power or requiring more LoRa gateways [14, 25, 26, 28]. On the other hand, if we can improve the LoRa demodulation process, enabling communication under lower SNR, we can considerably extend battery life and reduce deployment costs. Our problem is: can we design a better LoRa demodulation process to enable communication under lower SNR with an affordable overhead?

Status Quo and their Limitations. Table 1 displays the status quo approaches. [5, 7, 14, 35, 36] are non-neural-network methods. They use dechirp and FFT to condense signal energy, leveraging additional sources of information collected from additional gateways, nodes, etc. For instance, Charm [5]

condenses energy from 2 to 8 gateways, utilizing spatial diversity, XCopy [35] requires retransmission of LoRa signals to condense energy from multiple identical symbols, LoRaPhy [36] uses additional information provided by oversampling to better condense energy within each symbol, Choir [7] condenses energy from 10 to 30 additional co-located and synchronized transmitters, and MALoRa [14] condenses energy from 2 to 8 co-located and synchronized receivers. However, these methods require additional hardware, transmission power, or oversampling, inducing additional deployment cost and power consumption, influencing the wide scale and ubiquitous deployment of such systems.

Neural network-based methods like NeLoRa [17] use cognitive pattern recognition to distinguish signal from noise. Their overhead is high computational cost, requiring a dedicated computing platform for real-time decoding. Their method takes 7.19 to 32.24 ms to decode a single LoRa code with a PC with GPU, and running time increases to 13 to 120 seconds on a Raspberry PI. Furthermore, noise patterns are infinite and impossible to learn completely, so their gains are limited in real-life deployment (1.8 to 2.4 dB).

Overview of Our Design. In this paper, we propose LoRaTrimmer to solve the weak LoRa signal decoding from a new perspective. Previous methods' shortcomings include a lack of detailed analysis and observations of the LoRa signal itself. Non-neural methods are based on dechirp and FFT without exception; neural-based methods rely on neural networks to try their best to learn infinite noise patterns. Our approach is instead based on observations of the intrinsic nature of LoRa signals, and we found two critical problems in dechirp and FFT.

As shown in Fig. 1(a) and (b), ideal LoRa signals are "chirps" whose frequency increases linearly with time, and standard LoRa demodulation uses dechirp and FFT to condense power in these chirps (blue and gray stripes) into power peaks (red arrows). However, as shown in Fig. 1(c) and (d), the LoRa signals emitted by real-life transmitters contain a frequency jump. This jump breaks the energy of the symbol into two sections, which has two consequences: on the one hand, it

causes the signal duration (red line) to be shorter than the FFT perception range (blue stripe), inducing more noise than the signal and lowering SNR; on the other hand, it induces a random phase jump that causes a destructive add of the power. If we can mitigate these two problems caused by the frequency jump by redesigning the LoRa demodulation process, we can have significant SNR gains without overhead. However, designing such an algorithm is nontrivial. We face three main challenges:

- 1) The frequency jump breaks the signal into two segments of different frequencies, making the signal duration shorter than the FFT perception range, leading to additional noise induction. However, the time when the frequency jump happens varies with different symbols and is unknown before decoding. How can FFT stop inducing additional noise without knowing the exact time of the frequency jump?
- 2) The frequency jump caused a random phase jump that impacts the phase consistency of the symbol, causing signal power to add up destructively. An intuitive approach is to detect and measure the phase jump, but phase estimation becomes more inaccurate when SNR gets lower [12], drastically lowering the noise resistance of this method. Is there a way to add up power constructively without relying on knowledge of the phase jump?
- 3) LoRa gateways often have limited computational resources, but wireless communication requires real-time demodulation. How can a demodulation algorithm be designed to solve the previous two challenges and have the maximum SNR gain while maintaining a low computational profile?

To address the first challenge, our key design is to modify the FFT in the time domain. As shown in Fig. 1(d), we trim the perception range at different lengths according to its initial frequency, keeping the lengths of the perception ranges equal to the signal, mitigating this noise source. Although we do not know the exact time of the frequency jump, we see that it is related to the initial frequency of the LoRa signal. Thus, we can trim the signal unevenly, at a length that varies according to the initial frequency of the signal (see the difference in length of the blue and gray stripes). This way, no matter the LoRa signal, the FFT bin it falls in is trimmed to its exact length, completely mitigating the noise.

To address the second challenge, we observe that the phase jump and the initial frequency are two random variables that jointly determine the signal. Thus, the variable that encodes the data, the initial frequency, can be inferred by a posteriori analysis of the received signal. We condense the signal's power through a novel probabilistic modeling method, constructively adding power without needing the value of the phase jump. Our method can be proven under common assumptions to be the probabilistic optimal solution for decoding accuracy under low SNR.

To address the third challenge, we observe the similarity of our method with FFT and apply a similar divide-and-conquer strategy to reduce the computational cost from $O(n^2)$ to $O(n^{3/2})$, where n refers to the number of samples in each LoRa symbol. This is the only overhead of our method: a mere $O(n^{3/2})$ computational complexity compared with the $O(n \log n)$ of original FFTs. The computational time of our method is still at the millisecond level per packet, enabling real-time demodulation.

We have implemented an end-to-end system based on LoRaTrimmer, using USRP N210 [31] as the gateway and COTS LoRa radios (Semtech SX1276 [23]). We have deployed our gateways and LoRa nodes in various indoor and outdoor environments for evaluation. LoRaTrimmer achieves a gain of 1.70 to 2.49 dB SNR over the neural-based method NeLoRa [17], which is considered state-of-the-art, and a gain of 3.44 to 3.79 dB SNR over FFT-based methods. According to [28], a LoRa measurement study in urban environments, our SNR improvement can enlarge the coverage area of a single LoRa gateway from 11.4km² to 19.0km², an $1.67 \times$ gain.

Our only overhead is a slight increase in computation, at 0.51 to 3.17 ms per packet, compared with 0.23 to 0.94 ms of baseline methods at the same GPU server. Nonetheless, our method is capable of real-time decoding. LoRaTrimmer codes and datasets are open-sourced¹.

Our key contributions are threefold:

- We observe how the frequency jump in LoRa signals impacts low-SNR demodulation. On the one hand, it causes the signal to be shorter than the FFT power condensing range, inducing additional noise and lowering SNR; on the other hand, it induces a phase jump to cause destructive power addition.
- We observe that the induced additional noise can be distinguished from the signal from the time domain. Based on the relationships between the frequency jump time and the initial frequency of LoRa symbols, we modify the traditional dechirp+FFT approach by trimming the time perception range of FFT at different time locations for different frequencies, enabling us to completely cut off noise without knowing the exact time of the frequency jump.
- We re-evaluate the frequency jump in LoRa symbols, which impacts the phase consistency of the symbol. Through our signal-noise probabilistic analysis, we design a more efficient energy condensing and demodulation method (proved optimal under common assumptions), constructively adding up power from the whole signal without knowing the exact amplitude of the phase jump. Our method achieves an SNR gain of 1.70 to 2.49 dB SNR over the state-of-the-art, with the only overhead being a tiny increase in computation (still capable of real-time decoding).

¹Available at <https://github.com/LoRaTrimmer/LoRaTrimmer>.

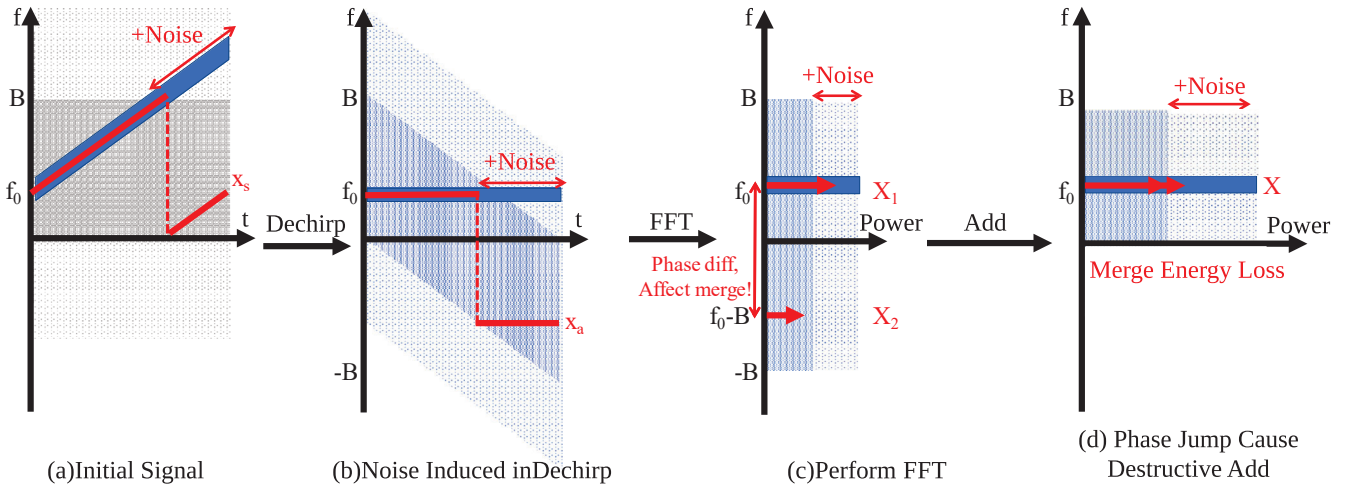


Figure 2: The standard LoRa dechirp+FFT method induces additional noise in the dechirp process and encounters power merging loss because of phase jump in the FFT process.

2 Background

In this section, we unveil the traditional LoRa demodulation process and discuss its fundamental limitations.

LoRa Symbols. Let the bandwidth be B and the spreading factor be SF . As shown in Fig. 2(a), a LoRa symbol is a chirp whose instantaneous frequency starts at an initial frequency f_0 ($0 \leq f_0 < B$), increases to B with a constant rate, jumps to 0, and then increases back to f_0 with the same constant rate. The data is encoded in f_0 , so the goal for demodulation is to derive f_0 of the LoRa symbol. The frequency jump from B to 0 will also induce a phase jump. Its time t_0 is given by:

$$t_0 = \left(1 - \frac{f_0}{B}\right)T, \quad (1)$$

where T indicates the symbol duration, which is defined by the spreading factor SF :

$$T = \frac{2^{SF}}{B}. \quad (2)$$

Denote upchirp $x_u(t)$ with initial frequency f and initial phase φ :

$$x_u(t; f, \varphi) := \exp\left(j2\pi\left(f + \frac{Bt}{T}\right)t + j\varphi\right), 0 \leq t < T \quad (3)$$

Let φ_0 be the initial phase of the received signal, φ_1 be the phase of the signal after the phase jump, A be the signal amplitude, define the received LoRa symbol without noise with initial frequency f_0 as $x_s(t)$:

$$x_s(t) = \begin{cases} Ax_u(t; f_0, \varphi_0), & \text{if } 0 \leq t < t_0 \\ Ax_u(t; f_0 - B, \varphi_1), & \text{if } t_0 \leq t < T \end{cases} \quad (4)$$

$\varphi_1 \neq \varphi_0$ indicates the phase jump. This phase jump exists universally on COTS LoRa nodes due to the constraints of

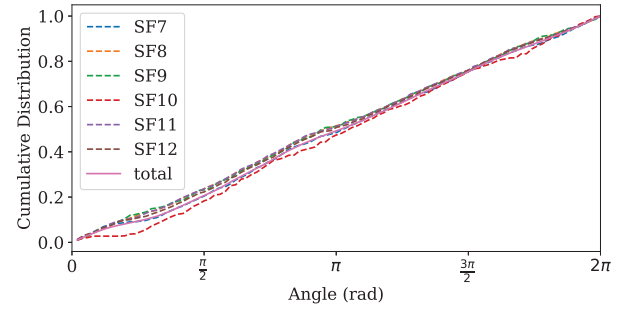


Figure 3: CDF of phase jump with 50 LoRa nodes and spreading factors from 7 to 12.

existing low-cost, low-power hardware. It is caused by hardware implementation to generate the frequency shift, and can be reduced by more expensive and power-consuming transmitters, e.g., using additional synchronous control circuits [8, 32], but this deviates from the purpose of LoRa to support cheap and low-power IoT devices.

To learn about the characteristics of the phase jump, we perform an empirical study, measuring signals from 50 LoRa nodes based on Semtech SX1276 and SX1268 [23] and with spreading factors from 7 to 12. For each SF configuration, we collect 10,000 symbols from each LoRa node. We calculate the phase jump of each symbol one by one. Fig. 3 shows the distribution of the phase jump values of all symbols. The phase jump exhibits an almost even distribution. Similar uniform distributions are observed across all SF settings, demonstrating its independence of symbol configurations.

Standard LoRa Demodulation Method. Let $x_n(t)$ be the noise, we can define the received LoRa symbol with noise

$x_n(t)$ as $x_r(t)$:

$$x_r(t) = x_s(t) + x_n(t) \quad (5)$$

There are three steps in the standard LoRa demodulation method. The first step is “dechirp”, which is to multiply the received signal with the conjugate of the reference upchirp $\bar{x}_u(t; 0, 0)$. Let the dechirped noise be $x'_n(t)$, and the result be $x_a(t)$:

$$\begin{aligned} x_a(t) &= x_r(t) \bar{x}_u(t; 0, 0) \\ &= \begin{cases} A \exp(j2\pi f_0 t + j\varphi_0) + x'_n(t), & \text{if } 0 \leq t < t_0 \\ A \exp(j2\pi(f_0 - B)t + j\varphi_1) + x'_n(t), & \text{if } t_0 \leq t < T \end{cases} \end{aligned} \quad (6)$$

The result will be two singletone signals at f_0 and $f_0 - B$, with initial phases φ_0 and φ_1 (as shown in Fig. 2(b)). The second step is to perform FFT on the signal. Let $X_n(f)$ be the FFT result of the noise $x'_n(t)$, and $\mathbb{1}_S$ be the indicator function of set S . FFT of $x_a(t)$ on $[0, t_0]$ is:

$$X_1(f) = A e^{j\varphi_0} \frac{t_0}{T} \mathbb{1}_{\{f_0\}}(f) + X_n(f), 0 \leq f < B, \quad (7)$$

and FFT of x_a on $[t_0, T]$ is:

$$X_2(f) = A e^{j\varphi_1} \frac{T - t_0}{T} \mathbb{1}_{\{f_0 - B\}}(f) + X_n(f), -B \leq f < 0, \quad (8)$$

The total FFT result should be $X_1(f) + X_2(f)$, exhibiting two peaks, as shown in Fig. 2(c). However, as shown in Fig. 2(d), since the bandwidth is only $[0, B)$, the two bins at f_0 and $f_0 - B$ are automatically overlapped, condensing the signal power into one single peak:

$$\begin{aligned} X(f) &= A \left(\frac{t_0}{T} e^{j\varphi_0} + \frac{T - t_0}{T} e^{j\varphi_1} \right) \mathbb{1}_{\{f_0\}}(f) \\ &\quad + X_n(f) + X_n(f - B), 0 \leq f < B. \end{aligned} \quad (9)$$

The final step is to estimate $\hat{f}_0$ by $\hat{f}_0 = \arg \max_f X(f)$, which decodes the symbol.

Problems. There are two problems with the standard LoRa demodulation method. The first problem is the unknown phase jump $\varphi_1 - \varphi_0$. In Eq. 9, the height of the signal bin $|X(f_0)| < A$ (setting $X_n(f)$ to 0) when $\varphi_0 \neq \varphi_1$, because the power of the two FFT bins $X_1(f_0)$ and $X_2(f_0 - B)$ add up destructively, causing severe loss of power. The power from the whole LoRa signal cannot be effectively condensed into one power peak, leading to decoding failure in low SNR. In an extreme case, when the phase jump is 180° and $t_0/T = 0.5$, $X(f_0)$ can even be zero, indicating the severe loss of signal power due to phase misalignment.

The second problem is the additional noise induced by dechirp. Although the signal bandwidth is $[0, B)$, the dechirp process (Eq. 6) changes it into $[-B, B)$, as shown in Fig. 2(b), which induces out of band noise from bandwidth $2B$ onto the B bandwidth signal. In other words, the perception range of FFT (the blue band) is longer than the signal segment, and the exceeding portion contains only noise, thus inducing

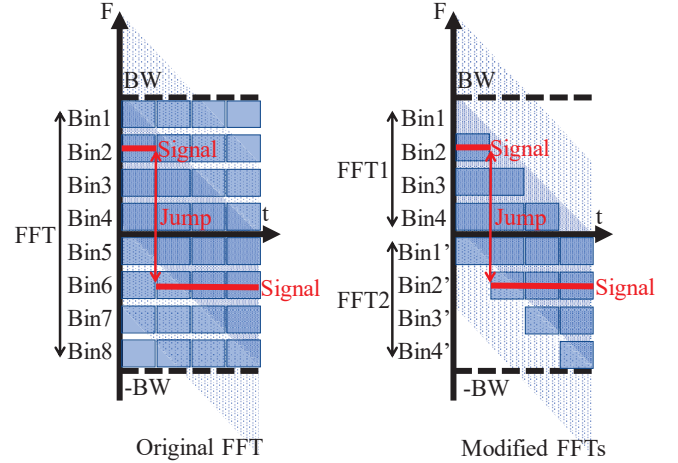


Figure 4: The design of our method, modifying FFT by trimming its perception range.

additional noise. In Eq. 7 and 8, signal only exists on $[0, t_0]$ for X_1 and $[t_0, T]$ for X_2 , but the FFT is performed on $[0, T]$ for X_1 and X_2 , which enables noise $X_n(f)$ to accumulate at full length T . This causes the noise power to double into $X_n(f) + X_n(f - B)$ at Eq. 9, as shown in Fig. 2(d), reducing the SNR by 3 dB. The intuitive solution is to use filters to remove the out-of-band noise, but this method has three limitations: (1) It requires oversampling the signal, which may not be available in practice; (2) An ideal filter requires infinite sampling rate, and because the ratio of sampling rate versus bandwidth (the oversampling rate) is often low, filters will be inefficient; and (3) In low SNR scenarios, e.g. -15 dB, noise power is much higher than signal power. In this case, a non-ideal filter will drastically distort the signal while failing to remove all noise, only partially solving the problem. We observe that this additional noise can be discriminated by the time domain. By trimming the perception range of FFT, we not only remove the additional noise but also separate the two portions of the signal. Our design is explained in the next section.

3 LoRa Decoding by LoRaTrimmer

3.1 Core Design

Based on the two problems observed in the previous section, additional noise and unknown phase jump, we make two main designs: (1) We modify the FFT process to trim the input signal at different time locations on the time domain for different FFT bins, and (2) We use probabilistic modeling to constructively add up power without knowing the exact value of the phase jump.

The first part, trimming the signal, is based on two key observations: First, the two optimizable factors, the phase

jump and the additional noise, can both be treated if we trim the signal (in the time domain) at the frequency jump time. On the one hand, this phase jump time separates the signal from additional noise, and trimming can completely remove that part of noise; on the other, the splitting of signals segregates the signal power before and after the phase jump, enabling a more efficient power merge scheme (Eq. *).

However, the frequency jump location is unknown. We then make the second key observation that the frequency jump location is related to the initial frequency (Eq. 1), and, thus, the index of the bin in the FFT. Thus, we can modify the FFT process to trim different bins differently, as shown in Fig. 4. We trim the signal at t_f , and perform two separate FFTs, one addressing $[0, t_f)$ and the other $[t_f, T)$, while the trimming position t_f is also related to the FFT frequency f by Eq. 1. The method simultaneously has two benefits: It limits the perception range of FFTs inside the signal band after dechirp, removing additional noise, and separates the signal precisely at the frequency jump, enabling more efficient subsequent handling of the phase jump.

This brings us to the second problem: the unknown phase jump. The features of this phase jump can be submerged in noise and hard to detect. We take another approach: Instead of trying to detect and measure the phase jump, we make a theoretical probabilistic analysis of the signals, whose result guides us to merge the results of the two FFTs with Eq. * without needing to know the phase jump, as described below. This achieves constructive addition of power, providing us with the theoretically highest SNR gain, while not requiring any pre-set parameters, ensuring robustness and generalizability in dynamic, complex IoT scenarios.

Specifically, assuming the notations in Sec. 1, given a received LoRa symbol $x_r(t)$, we calculate a value $X_v(f)$ for each of the 2^{SF} possible initial frequencies f ($0 \leq f < B$) by the following steps:

- (1) Dechirp the signal by multiplying it with a downchirp:

$$x_a(t) = x_r(t) \bar{x}_u(t; 0, 0) \quad (10)$$

- (2) Split the signal at t_f into x_{1f} and x_{2f} , the phase jump time based on f :

$$\begin{cases} t_f = \left(1 - \frac{f}{B}\right)T \\ x_{1f}(t) = x_a(t) \mathbb{1}_{[0, t_f)}(t) \\ x_{2f}(t) = x_a(t) \mathbb{1}_{[t_f, T)}(t) \end{cases} \quad (11)$$

- (3) Perform FFT on x_{1f} and x_{2f} , and pick the FFT bin at frequency f and $f - B$. Let the results be X_{1f} and X_{2f} , which is also denoted as FFT1 and FFT2 in Fig. 4. The frequency range of $x_{1f}(t)$ is $[0, B)$ and the signal is at f , while the frequency range of $x_{2f}(t)$ is $[-B, 0)$ and the signal is at $f - B$, so X_{1f} and X_{2f} are the signal

powers before and after the frequency jump.

$$\begin{cases} X_{1f} = \mathcal{F}\{x_{1f}(t)\}(f) \\ X_{2f} = \mathcal{F}\{x_{2f}(t)\}(f - B) \end{cases} \quad (12)$$

- (4) Compute $X_v(f)$ with the following equation:

$$X_v(f) = |X_{1f}|^2 + |X_{2f}|^2 \quad (*)$$

Repeat the above steps for all 2^{SF} initial frequencies f , and estimate $\hat{f}_0$ by $\hat{f}_0 = \arg \max_f X_v(f)$.

We now prove that under our theoretical model, this is the optimal method for low SNR LoRa demodulation.

3.2 Theoretical Analysis of LoRaTrimmer

LoRaTrimmer is deduced from a probabilistic theoretical analysis of LoRa symbols and noise models considering the phase jump. Note that LoRaTrimmer does not need oversampling to function; it is optimal at both Nyquist sampling and oversampling conditions. Oversampling occurs naturally in real-life scenarios where custom off-the-shelf hardware must support the highest bandwidth in LoRa specifications (500K) while lower bandwidths are often used (125K, 250K) for more noise resistance in low-SNR scenarios, which we are considering now. Our method not only solves the phase jump power loss but also makes the most of highly probable oversampling to remove additional noise, reaching the theoretical optimal in all scenarios.

We have two assumptions for our theoretical model:

ASSUMPTION 1. *The signal contains only Additional Gaussian White Noise (AWGN), and the received signal amplitude A and noise variance σ^2 remain steady (although unknown) throughout the signal.*

Let $x_r(t)$ be the received signal with noise, and $x_s(t)$, defined in Eq. 4 at Sec. 1, be the received signal without noise with amplitude A , initial frequency f_0 , initial phase ϕ_0 , and phase after jump ϕ_1 . Under Assumption 1, $x_r(t) - x_s(t)$ follows a Complex Normal Distribution at any t with variance σ^2 :

$$x_r(t) - x_s(t) \sim \mathcal{CN}(0, \sigma^2). \quad (13)$$

The probability density function (PDF) f of $|z|$ is:

$$f_{x_r(t)-x_s(t)}(|z|) = \frac{1}{\pi\sigma^2} \exp\left(-\frac{|z|^2}{\sigma^2}\right). \quad (14)$$

Based on an empirical study on the phase jump in Sec. 1, we make the following assumption:

ASSUMPTION 2. *ϕ_0 and ϕ_1 (the unknown initial phases before and after the phase jump) follow uniform independent distributions over $[0, 2\pi)$, also independent from f_0 .*

Let n be the number of samples in the symbol. Let $\{x_i\}_{i=0}^{n-1}$ be the series of received samples of the noisy signal, which are n random variables dependent on f_0 , the initial frequency

of the signal. To decode the received signal $\mathbf{x}_r(\mathbf{t})$, Our goal is to compute the initial frequency $\hat{f}_0$:

$$\hat{f}_0 = \arg \max_f P(\mathbf{x} = \mathbf{x}_r(\mathbf{t}) \mid f_0 = f) \quad (15)$$

for the time series $\mathbf{t}$ at all the sampling times $\{\mathbf{t}_i\}_{i=0}^{n-1}$.

By Bayes' theorem, Let $f_{\mathbf{x}}$ stand for PDF of $\mathbf{x}$:

$$P(\mathbf{x} = \mathbf{x}_r(\mathbf{t}) \mid f_0 = f) = \frac{f_{\mathbf{x}|f_0}(\mathbf{x}_r(\mathbf{t}) \mid f_0 = f)P(f_0 = f)}{f_{\mathbf{x}}(\mathbf{x}_r(\mathbf{t}))} \quad (16)$$

Since $P(f_0 = f) \equiv 1/2^{\text{SF}}$ for the 2^{SF} legitimate initial frequencies f , and $f_{\mathbf{x}}(\mathbf{x}_r(\mathbf{t}))$ does not change with f_0 , the problem becomes

$$\hat{f}_0 = \arg \max_f f_{\mathbf{x}|f_0}(\mathbf{x}_r(\mathbf{t}) \mid f_0 = f) \quad (17)$$

$$= \arg \max_f \prod_{i=0}^{n-1} f_{\mathbf{x}_i|f_0}(\mathbf{x}_r(\mathbf{t}_i) \mid f_0 = f) \quad (18)$$

By Eq. 13 and Assumption 2, $\hat{f}_0$ can be further reduced into:

$$\hat{f}_0 = \arg \max_f \prod_{i=0}^{n-1} \int_0^{2\pi} \int_0^{2\pi} d\varphi_0 d\varphi_1 \frac{1}{\pi\sigma^2} \exp\left(-\frac{|\mathbf{x}_r(\mathbf{t}_i) - \mathbf{x}_s(\mathbf{t}_i)|^2}{\sigma^2}\right) \quad (19)$$

Let Re stands for real part, Since

$$|\mathbf{x}_r(\mathbf{t}) - \mathbf{x}_s(\mathbf{t})|^2 = |\mathbf{x}_r(\mathbf{t})|^2 + |\mathbf{x}_s(\mathbf{t})|^2 - 2\text{Re}(\mathbf{x}_r(\mathbf{t})\overline{\mathbf{x}_s(\mathbf{t})}) \quad (20)$$

and $|\mathbf{x}_r(\mathbf{t})|^2$ is fixed, $|\mathbf{x}_s(\mathbf{t})|^2 = A$, we can derive

$$\hat{f}_0 = \arg \max_f \int_0^{2\pi} \int_0^{2\pi} \exp\left(\frac{2}{\sigma^2} \sum_{i=0}^{n-1} \text{Re}(\mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)})\right) d\varphi_0 d\varphi_1. \quad (21)$$

Apparently, based on Eq. 4:

$$\begin{aligned} \overline{\mathbf{x}_s(\mathbf{t})} &= \begin{cases} A\overline{\mathbf{x}_u(\mathbf{t}; -f, -\varphi_0)}, & \text{if } 0 \leq \mathbf{t} < \mathbf{t}_0 \\ A\overline{\mathbf{x}_u(\mathbf{t}; -f + B, -\varphi_1)}, & \text{if } \mathbf{t}_0 \leq \mathbf{t} < T \end{cases} \\ &= \begin{cases} A\overline{\mathbf{x}_u(\mathbf{t}; 0, 0)} \exp(-j2\pi f - j\varphi_0), & \text{if } 0 \leq \mathbf{t} < \mathbf{t}_0 \\ A\overline{\mathbf{x}_u(\mathbf{t}; 0, 0)} \exp(-j2\pi(f - B) - j\varphi_1), & \text{if } \mathbf{t}_0 \leq \mathbf{t} < T \end{cases} \end{aligned} \quad (22)$$

$\mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)}$ is comparable to the “dechirp” process on $\mathbf{x}_r(\mathbf{t})$:

$$\mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)} = \begin{cases} A\mathbf{x}_a(\mathbf{t}_i) \exp(-j2\pi f - j\varphi_0), & \text{if } 0 \leq \mathbf{t} < \mathbf{t}_0 \\ A\mathbf{x}_a(\mathbf{t}_i) \exp(-j2\pi(f - B) - j\varphi_1), & \text{if } \mathbf{t}_0 \leq \mathbf{t} < T \end{cases} \quad (23)$$

Due to the fact that the sum of a signal is equivalent to the FFT bin at 0 frequency, let p indicate the index for phase jump: $\mathbf{t}_p = \mathbf{t}_0$ in Eq. 1:

$$\begin{cases} \sum_{i=0}^{p-1} \mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)} = A\mathbf{e}^{j\varphi_0} \mathcal{F}\{\mathbf{x}_{1f}(\mathbf{t})\}(f) \\ \sum_{i=p}^{n-1} \mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)} = A\mathbf{e}^{j\varphi_1} \mathcal{F}\{\mathbf{x}_{2f}(\mathbf{t})\}(f - B) \end{cases} \quad (24)$$

$$\sum_{i=0}^{n-1} \mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)} = AX_{1f}\mathbf{e}^{j\varphi_0} + AX_{2f}\mathbf{e}^{j\varphi_1} \quad (25)$$

$$\begin{aligned} \text{Re}\left(\sum_{i=0}^{n-1} \mathbf{x}_r(\mathbf{t}_i)\overline{\mathbf{x}_s(\mathbf{t}_i)}\right) &= A|X_{1f}| \cos(\varphi_0 + \theta_{1f}) \\ &\quad + A|X_{2f}| \cos(\varphi_1 + \theta_{2f}) \end{aligned} \quad (26)$$

Where θ_{1f} is phase for X_{1f} , θ_{2f} is phase for X_{2f} .

From Eq. 21, use I_0 to denote the Modified Bessel Function of the First Kind of order zero, and using the fact that

$$\int_{-\pi}^{\pi} \exp(a \cos \theta) d\theta = 2\pi I_0(a), \quad (27)$$

$$\hat{f}_0 = \arg \max_f I_0\left(\frac{2A|X_{1f}|}{\sigma^2}\right) I_0\left(\frac{2A|X_{2f}|}{\sigma^2}\right) \quad (28)$$

Since A is the signal amplitude and σ^2 is the noise power, A/σ^2 is the signal-to-noise ratio (SNR). At low SNR, $A/\sigma^2 \rightarrow 0^+$, we have

$$I_0(A'x) = 1 + \frac{A'^2 x^2}{4} + O(A'^2), \quad (29)$$

Eq. 28 can be approximated at low SNR by:

$$\hat{f}_0 = \arg \max_f |X_{1f}|^2 + |X_{2f}|^2 \quad (30)$$

This is Eq. *, the method's correctness is proved.

Conclusion. From the theoretical analysis, we prove the correctness of the method provided in Sec. 3.1. The method is used when we do not know the phase jump and the SNR. If any one of them, or both, are known (e.g. from hardware characteristics or previous measurements), one can easily replace the unknown values with these measurements. If the phase jump distribution is measured for specific hardware and is not uniform, one can add it in Eq. 19 and integrate the PDF of $\varphi_1 - \varphi_0$. If SNR is known, one can use Eq. 28 to compute $\hat{f}_0$. The presented method removes the need for knowing or trying to measure the values of SNR or the phase jump, providing a robust and efficient LoRa demodulation method with zero parameters and no need for any measurements and parameter tuning in advance.

3.3 Algorithm Complexity Optimization

The method proposed in Sec. 3.1 can be implemented by matrix multiplication. The Discrete Fourier Transform (DFT) matrix is

$$W := \left(\frac{\omega^{xy}}{\sqrt{n}} \right)_{x,y=0,\dots,n-1}, \quad (31)$$

with $\omega = e^{j2\pi/n}$ is a primitive n^{th} root of unity. Because our method splits the signal at $t_f = (1 - f/B)T$ for frequency f , our method can be simplified as

$$\hat{f}_0 = \frac{2^{\text{SF}}}{B} \arg \max_i (|Ux_a(t)|^2 + |Dx_a(t)|^2) \quad (32)$$

and U and D are the upper and lower antitriangles of W :

$$U := \left(\frac{\omega^{xy}}{\sqrt{n}} \mathbb{1}_{[0,n)}(x+y) \right)_{x,y=0,\dots,n-1}, \quad (33)$$

In other words,

$$U = \frac{1}{\sqrt{n}} \begin{bmatrix} \omega^{0 \cdot 0} & \dots & \omega^{0 \cdot (n-2)} & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \dots & \omega^{1 \cdot (n-2)} & 0 \\ \vdots & \ddots & \vdots & \vdots \\ \omega^{(n-1) \cdot 0} & \dots & 0 & 0 \end{bmatrix}, \quad (34)$$

$$D := \frac{1}{\sqrt{n}} (W - U) \quad (35)$$

Assume $n = 2^l$, $l \in \mathbb{N}^+$, which is natural due to the length of LoRa symbols according to Eq. 2. Define q as a close resemblance to $\sqrt{n}$: $q = 2^{\lfloor l/2 \rfloor}$. We now show how to reduce time complexity to $O(n^2/q)$, hence $O(n^{3/2})$, and memory complexity to $O(n)$ through divide and conquer.

Given an array $\{x_a(t_i)\omega^{ri}\}_{i=0}^{n-1}$, select m numbers starting from s with step k and denote their sum as $S(r, s, k, m)$:

$$S(r, s, k, m) := \sum_{i=0}^{m-1} x_a(t_{s+ik}) \omega^{r(s+ik)} \quad (36)$$

Thus, the r^{th} component of $Ux_a(t)$ is: ($0 \leq r < n$)

$$U_r x_a(t) = \sum_{i=0}^{n-r-1} x_a(t_i) \omega^{ri} = S(r, 0, 1, n-r), \quad (37)$$

which can be transformed into:

$$S(r, 0, 1, n-r) = \sum_{k=0}^{q-1} S\left(r, k, q, \left\lceil \frac{n-r-k}{q} \right\rceil\right). \quad (38)$$

Furthermore, for any integer $y \in [0, q)$, if $r \in [0, n/q)$, using the fact that $\omega^n = 1$:

$$S\left(r + y\frac{n}{q}, k, q, \left\lceil \frac{n-r-k}{q} \right\rceil\right) = S\left(r, k, q, \left\lceil \frac{n-r-k}{q} \right\rceil\right) \omega^{\frac{yn}{q}} \quad (39)$$

Because $n/q^2 = 1$ or 2 , we can replace r with $r + yn/q$ in Eq. 38:

$$\begin{aligned} S\left(r + y\frac{n}{q}, 0, 1, n-r-y\frac{n}{q}\right) &= \sum_{k=0}^{q-1} S\left(r + y\frac{n}{q}, k, q, \left\lceil \frac{n-r-k-y\frac{n}{q}}{q} \right\rceil\right) \\ &= \sum_{k=0}^{q-1} S\left(r + y\frac{n}{q}, k, q, \left\lceil \frac{n-r-k}{q} \right\rceil - \frac{yn}{q^2}\right) \end{aligned} \quad (40)$$

This and Eq. 39 proves that $S(r + yn/q, k, q, \left\lceil \frac{n-r-k-yn/q}{q} \right\rceil)$ is a prefix sum of the sum $S(r, k, q, \left\lceil \frac{n-r-k}{q} \right\rceil)$ times a factor $\omega^{\frac{yn}{q}}$, so for $y = 0, \dots, q-1$, the q terms of $S(r + yn/q, k, q, \left\lceil \frac{n-r-k-yn/q}{q} \right\rceil)$ can be computed together with $O(\left\lceil \frac{n-r-k}{q} \right\rceil) = O(n/q)$ time. In Eq. 37 and Eq. 38, $Ux_a(t)$ contains $n \times q$ terms of $S(r + yn/q, k, q, \left\lceil \frac{n-r-k-yn/q}{q} \right\rceil)$, so they can be computed in $O(n^2/q)$, hence $O(n^{3/2})$ time. Furthermore, this computation can be done in place, computing q terms each time and adding them to the corresponding rows, so the whole computation only requires $O(n)$ memory.

The n used in complexity analysis is the number of samples per LoRa symbol. Under the same bandwidth, the symbol rate is increased by reducing the spreading factor SF, which shortens the symbol length, so when the symbol rate is doubled, n is halved, and computational cost per symbol is thus reduced to $(1/2)^{3/2} = \sqrt{2}/4$. When the bandwidth B doubles, the sampling rate is also doubled to meet the Nyquist criterion, but the symbol length is shortened by half, so n and computational cost per symbol does not change if we use the Nyquist sampling rate. If the sampling rate is fixed, doubling bandwidth leads to the symbol length being shortened by half, so the computational cost per symbol is reduced to $\sqrt{2}/4$.

4 Packet Detection

LoRa packets consist of two main parts: preamble and payload [16, 20]. The preamble contains multiple base upchirps, used for packet detection, and 2.25 base downchirps called Start of Frame Delimiter(SFD), used for offset correction.

During packet detection, we aim to detect the existence and position of the consecutive base upchirps. For an input sequence of $k \times n$ samples (k is an integer) that might contain a LoRa packet, we run a sliding window of length $2n$ on the collected data, with the sliding step of n , generating $k-1$ windows. If a certain window contains part of the preamble, there should be one complete upchirp in it. To detect it, we multiply the window with the concatenation of U and D :

$$X = [U \quad D]x_a(t) \quad (42)$$

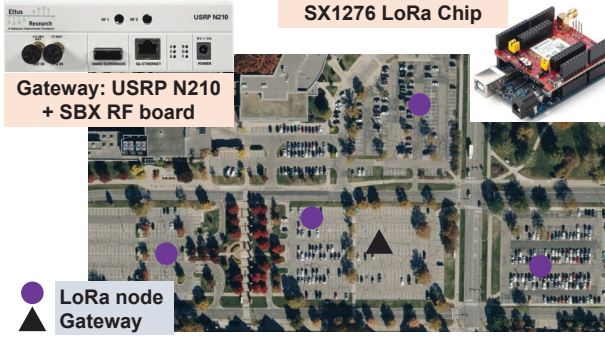


Figure 5: Hardware implementation of LoRaTrimmer.

If an upchirp is in the window, there will be a power peak. The consecutive upchirps will generate consecutive peaks with the sliding window with the same position and phase. From here, we can use methods similar to NeLoRa [17] to add up power from these consecutive peaks to detect the presence of the preamble, and compute and remove time and frequency offsets from the SFD. Now, we can split the payload into symbols and decode them. We now construct an end-to-end LoRa demodulation system, and we will implement and evaluate it in the next section.

5 Implementation and Evaluation

5.1 Hardware Implementation

We implement LoRaTrimmer with Semtech SX1276 [23] LoRa nodes and a LoRa gateway using a ThinkPad E460 laptop and a USRP N210 SDR [31] with SBX 400-4400 RF daughterboards [30]. We use UHD+GNU-radio software for control. The sampling rate is 1 MHz, and experiments are carried out at 433 MHz central frequency, bandwidth 125 KHz, SF 7 to 12 (default at 7), and preamble containing 8 to 64 upchirps (default at 8). We deploy our system indoors and outdoors. The indoor environment is our office full of furniture and computers, where adequate human activity induces dynamic environmental changes. The outdoor environment is in a parking lot. We add synthetic noise to the data to generate signals with different SNRs for evaluation.

Moreover, due to the low complexity of our method, it is also possible to use TinySDR [13], consisting of low-cost radio front-end, FPGA, and microcontroller, to reduce the cost (\$55) and energy consumption (mW-level). LoRaTrimmer is in the physical layer, with an ms-scale processing delay, so it is flexible in scaling the number and capability of RF chains to support concurrent decoding and high data rates.

5.2 Methods for Comparison

Since our default sampling rate of 1 MHz is larger than the bandwidth of 125 KHz due to oversampling, the result of

direct dechirping and FFT will contain two bins. We compare our method with the following methods:

Multiple FFT(MFFT): This method breaks the samples into multiple signals with sampling rate B , compensating for their phase difference and adding their FFT results [33];

Highest FFT Bin(HFFT): This method decodes based on the position of the highest FFT bin in the range $-B/2$ to $B/2$, only accounting for the higher of the two bins;

LoRaPhy: The work from [36] provides two methods: “Abs+” adds the absolute values of two bins (each pair of bins with distance B between $-B$ and B) into one and finds the bin of the highest power; Search Phase and Add(“Ps+”) searches for the value of the phase difference between the two bins assuming that if the phases are aligned, the maximum added power will be the highest and uses this phase difference to add the two bins constructively. However, a fine-grained phase search is equal to that of the Abs+ method. Once the sum of absolute values of a pair of noise peaks overwhelms the signal peak, they will also dominate the phase-search process, generating a phase to make the noise peaks coherently added. Thus, the height and position of the highest bin of Abs+ and Ps+ are always the same (differences caused by insufficient search granularity and are completely random), so we plot the two methods together as LoRaPhy.

NeLoRa: This method does not involve either dechirping or FFT processes but instead applies a neural network on the spectrogram to classify symbols. Here, NeLoRa requires high computational cost, and the model fails to converge at SF > 10 , so we cannot show its performance at SF = 11 and SF = 12. This is mainly due to the rapid increase in the number of classes for the neural network to classify (4096 at SF = 12), which requires an exponentially larger model width to perform this classification (which is also admitted and not implemented in the original paper [17]).

LoRaTrimmer and “Opt”: We implement LoRaTrimmer without knowing the SNR, and assume SNR $\rightarrow 0$, using Eq. * in Sec. 3.1. We also provide the theoretically optimal version “Opt”, where the SNR is accurately known (hypothetically through listening to the preamble and empty wireless channel), and Eq. 28 in Sec. 3.2 is used to demodulate the symbol.

5.3 Overall Performance across SNR and SF

We compare the decoding accuracy of all methods at different SNRs at SF from 7 to 12. The results of SF at 7 and 10 are shown in Fig. 6 and Fig. 7. We use the SNR at accuracy 90% as a threshold and calculate the SNR thresholds of different methods at different SF, shown in Fig. 8. LoRaTrimmer achieves 1.70 to 2.49 dB SNR gain over NeLoRa and 3.44 to 3.79 dB SNR gain over FFT-based methods at 90% accuracy threshold at SF from 7 to 12. The HFFT, LoRaPhy, and MFFT methods are based on FFT and yield similar results; their

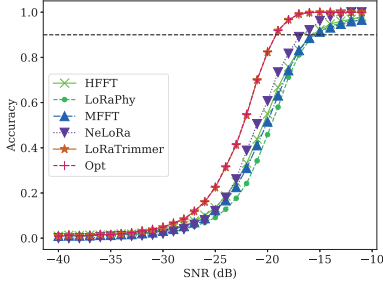


Figure 6: Decoding accuracy at different SNR, SF = 7.

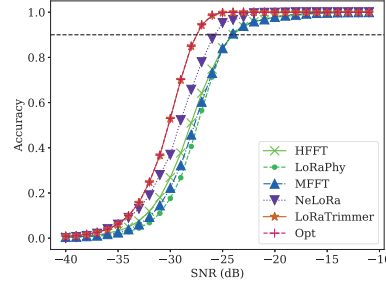


Figure 7: Decoding accuracy at different SNR SF = 10.

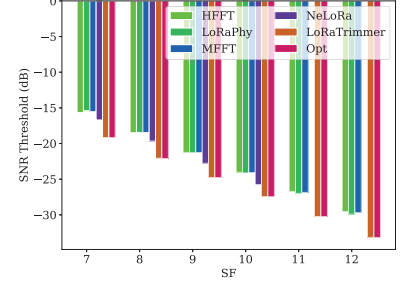


Figure 8: SNR thresholds of different methods, SF from 7 to 12.

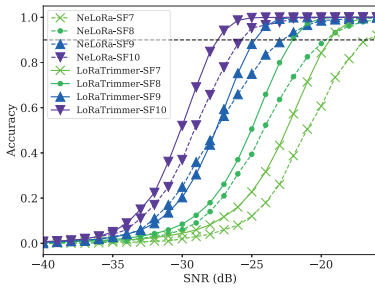


Figure 9: Decoding accuracy compared to NeLoRa.

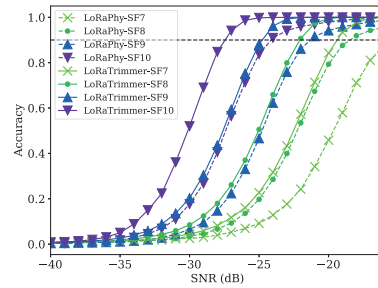


Figure 10: Decoding accuracy compared to LoRaPhy.

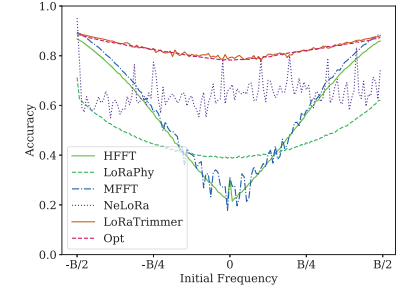


Figure 11: Accuracy at different symbols.

performances are all influenced by the phase jump problem. NeLoRa achieved some improvements over these three methods since it directly processes the spectrogram as an image with powerful neural networks, partially bypassing the phase jump problem, but the spectrogram, when viewed as an image, cannot effectively condense the power of the whole signal into a single peak, so it's challenging for neural networks to reach the optimal accuracy, and it's also computational costly. Our method LoRaTrimmer approaches the optimal accuracy by accurately modeling the signal and considering the phase jump, resulting in significant SNR gains with very limited computational overhead. "Opt" is used as a benchmark for the theoretical upper bound. Fig. 8 shows that "Opt" achieves 0.005 to 0.016 dB gain over LoRaTrimmer, and in Fig. 6, Fig. 7, we can see that "Opt" almost overlaps with LoRaTrimmer, showing that our method has already approximated the theoretical upper bound and the need for the SNR hyperparameter is trivial.

5.4 Accuracy Comparison across SFs

We compare the performance of our method with NeLoRa and LoRaPhy in Fig. 9 and Fig. 10 respectively, at different SNRs and SF from 7 to 10. As we stated, LoRaPhy has similar performance with MFFT and HFFT and we select it because it

has the best overall performance among these three and is believed to be the method used in the commercial closed-source devices [36]. Fig. 9 and Fig. 10 show that we are superior to these methods at all SNRs and SFs.

5.5 Accuracy for Different Symbols

Our core observation is that the phase jump influences noise condensing in the LoRa dechirp and FFT process and thus affects decoding. This affects different symbols at different degrees. At symbols whose initial frequencies f_0 is close to 0 and B , the phase jump is at the start and end of the signal, and the majority of the signal remains phase-continuous, suffering less from the phase jump; when f_0 close to $B/2$, the phase jump occurs at the middle of the signal, splitting the power into two halves, and if they are not constructively added, the decoding accuracy will fall severely. In Fig. 11, we compare the performance of the methods at different symbols at SF = 7 and SNR = -20 dB. We can see that methods HFFT and MFFT are influenced most severely, exhibiting a fast decrease and increase in accuracy as f_0 changes. This verifies our claim that power is split at the phase jump. HFFT only selects the highest peak, and MFFT directly adds up power from the two peaks regardless of phase difference (so power adds up destructively), so they are influenced the most.

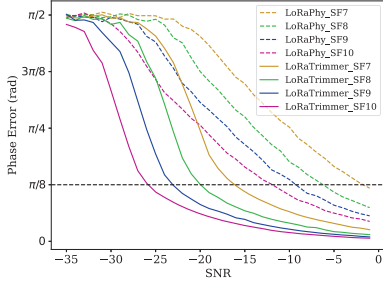


Figure 12: Comparison of phase estimation errors.

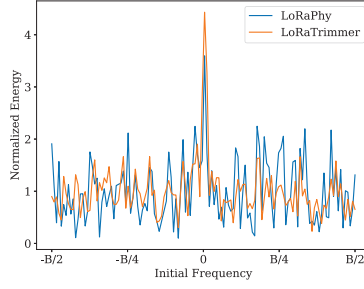


Figure 13: Shape of our peak compared to method LoRaPhy.

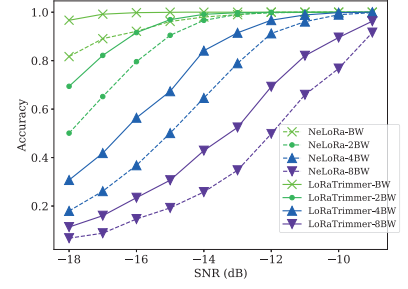


Figure 14: Performance in various sampling rates compared to NeLoRa.

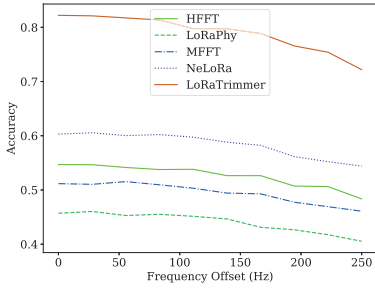


Figure 15: Performance under manually induced CFO.

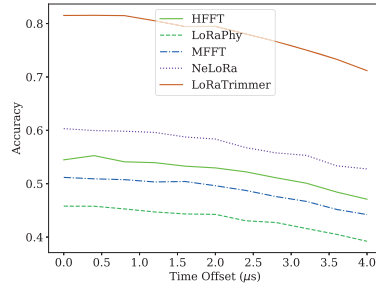


Figure 16: Performance under manually induced Time Offsets.

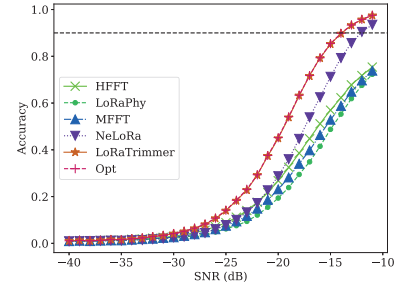


Figure 17: Accuracy with constant interference.

The result of the addition in MFFT depends highly on the random phase jump, so its results are much more unstable. There is a slight increase of HFFT at $f_0 = B/2$. This is due to the frequency offsets, and for $f_0 = B/2$, the two peaks are equally possible to be shifted by offsets to be partially within the range of 0 to B . LoRaPhy uses phase estimation and energy-adding processes to partially mitigate the accuracy drop when f_0 is close to $B/2$, but since they cannot correctly estimate the phase jump under low SNR, they still exhibit different performances at different symbols. Due to the errors in estimation, their accuracy is even lower than MFFT and HFFT when f_0 is close to 0 and B . This method performs better at $f_0 = 0$ due to the absence of phase jump. NeLoRa does not suffer from this effect since it directly operates on a spectrogram and does not involve the dechirp and FFT process. Nonetheless, it also has uneven decoding performance on different symbols and achieves remarkably higher accuracy at $f_0 = 0$ since the symbol does not involve phase jump and exhibits a continuous power band in the spectrogram which is easier to detect for the local receptive field of CNNs. However, the neural network needs to learn the features of all 2^{SF} images in order to make a correct classification, which leads to instability across the symbols and the exponential increase of required network size when

increasing SF to larger values. In comparison, our methods (both LoRaTrimmer and “Opt”) gain even and high decoding accuracy on all symbols, fully mitigating the influences of phase jump even without estimating its exact value. Note that “Opt” is the probabilistic optimal solution, so LoRaTrimmer may surpass “Opt” in some symbols due to probabilistic fluctuations.

5.6 Peak Shape and Phase Estimation

Fig. 12 shows the comparison of phase estimation error compared with LoRaPhy [36]. We can see that when error is at $\pi/8$, our SNR gain is 13.7 to 14.3 dB at different spreading factors. This verifies that LoRaPhy searches for the phase that can constructively add signal power, but their phase search is often error-prone under low SNR. LoRaTrimmer is not only better at low-SNR decoding but also provides a much more accurate phase estimation.

Fig. 13 selects a random sample from the theoretical packets and compares the resulting power peaks of LoRaTrimmer and the results of LoRaPhy when decoding them. The energy is normalized by mean. Although LoRaPhy designs to generate undistorted peaks, LoRaTrimmer generates a more clarified peak with lower noise peaks at other bins. The superior performance on phase estimation and peak clarity prove

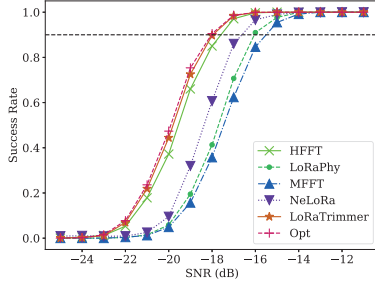


Figure 18: Packet detection rate at 8 preambles with SF = 7.

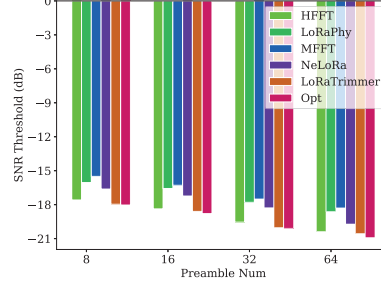


Figure 19: SNR threshold at different number of preambles with SF = 7.

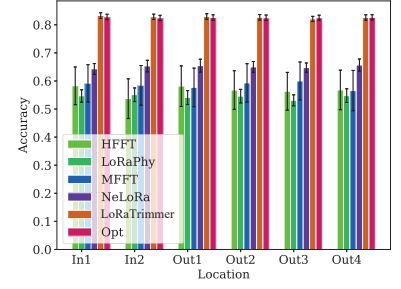


Figure 20: Accuracy at different environments under same SNR.

that our method can be expanded to other LoRa applications, e.g., in localization and collision recovery, where distinguishing multiple peaks and measuring phases is critical.

5.7 Performance Across Sampling Rates

We evaluate the decoding performance of LoRaTrimmer at different sampling rates from B to $8B$, as shown in Fig. 14, and compare it with NeLoRa. At all sampling rates, our performance is significantly higher than NeLoRa's. Even without oversampling, at sampling rate B , there is still a gain of 0.83 to 1.12 dB over NeLoRa, demonstrating our superiority.

5.8 Resistance Against CFO and TO

CFO and TO can be induced by environmental factors like mobility-induced Doppler shifts. To evaluate the resistance of our method against offsets, we manually induce different levels of CFO and TO in the packet alignment process. The results shown in Fig. 15 and Fig. 16 demonstrate that the accuracy of LoRaTrimmer for such frequency and time shifts drops from 11.9% to 13.7% compared to the offset-free value, while NeLoRa and other FFT-based methods drop from 9.2% to 12.6%. We did not make any assumption about signal errors in our method, so the drop rate is slightly higher. However, in practical settings, the offsets will be mitigated by the alignment process and will be much smaller than what is shown in the figures, and even after the drop, the performance of our method is still significantly better.

5.9 Resistance Against Interference

Varying degrees of interference can exist in the LoRa transmission channel. To test the resistance of LoRaTrimmer against interference, we measured the decoding accuracy under different SNRs by inducing additional interference to the signals. We use another USRP to generate four kinds of interference: singletone, pulse, LoRa packets with different SFs, and LoRa packets with the same SF. The USRP is placed

next to the LoRa transmitting node, providing an interference source with a constant Signal Interference Ratio(SIR). Then we adjust the SNR and measure the decoding accuracy of different methods at the same SIR and different SNRs. The results are shown in Fig. 17. NeLoRa inherently has a higher resistance to interference since it processes the spectrogram directly and can identify the signal from interference. However, LoRaTrimmer still has an 1.85 dB gain over NeLoRa, demonstrating our robustness.

5.10 Robustness in Different Environments

As described in Sec. 5.1, we tested our method at different locations, indoors and outdoors, with different LoRa transmitters. Fig. 20 displays the results of multiple experiments at different locations under the same SNR. Our method evenly improve the decoding accuracy at all codes and under all noise patterns, exhibiting high accuracy and robustness across different devices and environments.

5.11 Packet Detection Rate

As introduced in Sec. 4, LoRaTrimmer can be adopted to improve packet detection under low SNR. We use different numbers of upchirps: 8, 16, 32, and 64 in the preamble to enhance the packet detection rate at SF = 7. We use the same standard for packet detection across the methods for comparison, searching for a fixed number of periodic peaks with no more than one bin offset for a packet to be detected. The packet detection accuracy is shown in Fig. 18, and the 90% packet detection rate threshold is shown in Fig. 19. LoRaTrimmer induces an SNR gain of 0.45 to 0.55 dB compared to HFFT, the second-best method. This is because the packet detection process aims to detect the consecutive base upchirps in the preamble. These upchirps do not contain phase jump, and in this case, HFFT's performance is similar to ours. However, HFFT's performance is far lower than ours in decoding because the payload part of a LoRa packet consists of chirp symbols with random codes, which are significantly

Table 2: Computational time for demodulating a packet with 16 chirp symbols for different SF: unit: ms

Method	SF7	SF8	SF9	SF10	SF11	SF12
NeLoRa	7.19	9.05	12.34	32.2	N/A	N/A
LoRaPhy	1.24	2.43	3.20	4.50	5.72	7.11
HFFT	0.34	0.46	0.54	0.63	0.73	1.01
MFFT	0.23	0.30	0.48	0.57	0.80	0.94
LoRaTrimmer	0.51	0.76	1.25	1.94	2.51	3.17

influenced by phase jumps in decoding. In such cases, our method greatly outperforms HFFT by mitigating the influence of the phase jumps in decoding.

5.12 Computational Cost

We evaluate the computational cost of LoRaTrimmer and other baseline methods when decoding a 16-symbol LoRa packet on a PC with AMD Ryzen 9 5950X CPU and one Nvidia 2080 Ti GPU. The results are shown in Table 2. The processing time of our method is significantly shorter than the total length of these symbols, so our method can efficiently achieve real-time demodulation. Our method is not only significantly faster than the previous state-of-the-art NeLoRa, even when favoring them by using GPU, but also faster than LoRaPhy, because LoRaPhy contains time-consuming filtering and phase search processes. The baseline methods HFFT and MFFT contain only dechirp and FFT processes, so they are faster than our method. However, our method costs a slight increase in computational overhead to largely improve demodulation accuracy. We still keep the computation overhead at the millisecond level to ensure real-time decoding.

6 Related Work

From the transmitter side, Choir [7] leverages co-located LoRa nodes transmitting correlated packets to enlarge the communication range, and XCopy [35] coherently combines retransmitted packets to improve SNR. On the receiver side, Charm [5] mixes and coherently combines received signals from multiple gateways to detect weak signals that cannot be decoded at any individual gateway by reinforcing the peaks of correlated data symbols in one frequency bin and detecting them. Furthermore, Chime [11] minimizes multipath interference and saves battery energy of LPWAN by analyzing the wireless channels of one sent packet across many base stations. Nephelai [22] jointly demodulates samples acquired by compressive sensing in the cloud with a sparse approximation. Compared to LoRaTrimmer, all these works require multiple pairs of transceivers or multiple transmissions.

Some previous works have conducted theoretical analysis on LoRa decoding performance in different fading channels and antenna polarization [2, 4, 9, 10, 15, 29, 34], Demeslay et al. [3] perform theoretical analysis on multipath channels, and other works [18, 19, 27] analyze modulated symbol

shapes. However, they all fail to account for the phase jump inside the frequency jump by LoRa symbols generated by real hardware. In oversampling scenarios, Savaux [33] presents a method to demodulate the oversampled LoRa signal by applying multiple FFTs on odd and even samples, but the phase jump still affects each FFT. Xu et al. [36] presents the oversampling-based phase-aligned dechirp that mitigates the SNR loss due to the phase difference resulting from the frequency jump in each symbol. Their method requires oversampling to function, and only functions at high SNR, because at low SNR, the phase of FFT peaks will be influenced by noise, and they cannot get an accurate estimation. In comparison, our method optimally combines the power of the two parts of the signal without relying on the exact values of the phase difference, which not only removes the oversampling requirement but also generates significant SNR gains.

In addition to approaches based on physical models, deep learning techniques have been adopted to enhance LoRa communication. NELoRa [17] proposes a general LoRa demodulator based on DNN instead of dechirp that achieves a lower SNR threshold with acceptable computation overhead at the gateway side. Compared to NeLoRa, LoRaTrimmer does not require training on large datasets of LoRa packets while achieving higher SNR gain and requires significantly less computational cost.

7 Conclusion

In this paper, we observe that the frequency jump inside LoRa chirp symbols not only induces additional noise to the original FFT but also generates a phase jump that causes a destructive add-up of power, causing a loss of signal power and reducing communication range. We propose LoRaTrimmer, a theoretically optimal LoRa decoding method. We first trim the FFT perception range to remove the additional noise, and then merge the power with a probabilistic modelling method without needing the value of the phase jump. Experiments in complex real-life environments show that LoRaTrimmer achieves 1.70 to 2.49 dB SNR gain over the state-of-the-art and 3.44 to 3.79 dB SNR gain over FFT-based methods, translating to at most 1.67 times gain of coverage area. Our method is robust to all kinds of misalignments and interference and capable of real-time decoding. Our only overhead is a slight increase in computational cost: 0.51 to 3.17 ms per packet, compared with 0.23 to 0.94 ms of baseline methods, still supporting real-time symbol decoding.

8 Acknowledgement

This work is supported in part by National Key R&D Program of China under grant 2021YFB2900100, National Natural Science Foundation of China under grant 62332012.

References

- [1] LoRa Alliance. 2024. LoRaWAN Specification. <https://loro-alliance.org/about-lorawan>. Accessed 11-Aug-2024.
- [2] Adrian M. Deaconu and Javad Tayyebi. 2024. Increasing the Maximum Capacity Path in a Network and Its Application for Improving the Connection Between Two Routers. *Tsinghua Science and Technology* 29, 3 (2024), 753–765. <https://doi.org/10.26599/TST.2023.9010055>
- [3] Clément Demeslay, Philippe Rostaing, and Roland Gautier. 2022. Theoretical Performance of LoRa System in Multipath and Interference Channels. *IEEE Internet of Things Journal* 9, 9 (2022), 6830–6843.
- [4] Claudio Ferreira Dias, Eduardo Rodrigues de Lima, and Gustavo Fraidenraich. 2019. Bit Error Rate Closed-Form Expressions for LoRa Systems under Nakagami and Rice Fading Channels. *Sensors* 19, 20 (2019), 4412.
- [5] Adwait Dongare, Revathy Narayanan, Akshay Gadre, Anh Luong, Artur Balanuta, Swarn Kumar, Bob Iannucci, and Anthony G. Rowe. 2018. Charm: exploiting geographical diversity through coherent combining in low-power wide-area networks. In *Proceedings of ACM/IEEE IPSN*.
- [6] Jialuo Du, Yidong Ren, Zhui Zhu, Chenning Li, Zhichao Cao, Qiang Ma, and Yunhao Liu. 2023. SRLoRa: Neural-enhanced LoRa Weak Signal Decoding with Multi-gateway Super Resolution. In *Proceedings of ACM MobiHoc*.
- [7] Rashad Eletreby, Diana Zhang, Swarn Kumar, and Osman Yagan. 2017. Empowering Low-Power Wide Area Networks in Urban Settings. In *Proceedings of ACM SIGCOMM*.
- [8] Tallal Elshabrawy, Phoebe Edward, Mohamed Ashour, and Joerg Robert. 2019. On the different mathematical realizations for the digital synthesis of LoRa-based modulation. In *Proceedings of European Wireless*.
- [9] Tallal Elshabrawy and Joerg Robert. 2018. Closed-Form Approximation of LoRa Modulation BER Performance. *IEEE Communications Letters* 22, 9 (2018), 1778–1781.
- [10] Menno Jan Faber, Klaas Minne van der Zwaag, Willian Garcia Viegas dos Santos, Helder Roberto de O. Rocha, Marcelo E. V. Segatto, and Jair A. L. Silva. 2020. A Theoretical and Experimental Evaluation on the Performance of LoRa Technology. *IEEE Sensors Journal* 20, 16 (2020), 9480–9489.
- [11] Akshay Gadre, Revathy Narayanan, Anh Luong, Anthony G. Rowe, Bob Iannucci, and Swarn Kumar. 2020. Frequency Configuration for Low-Power Wide-Area Networks in a Heartbeat. In *Proceedings of USENIX NSDI*.
- [12] Chaojie Gu, Linshan Jiang, and Rui Tan. 2019. LoRa-Based Localization: Opportunities and Challenges. In *Proceedings of EWSN*.
- [13] Mehrdad Hesar, Ali Najafi, Vikram Iyer, and Shyamnath Gollakota. 2020. TinySDR: Low-Power SDR Platform for Over-the-Air Programmable IoT Testbeds. In *Proceedings of USENIX NSDI*.
- [14] Ningning Hou, Xianjin Xia, and Yuanqing Zheng. 2023. Don't Miss Weak Packets: Boosting LoRa Reception with Antenna Diversities. *ACM Transactions on Sensor Networks* 19, 2 (2023), 41:1–41:25.
- [15] Yingdan Jiang, Zongguang Yu, Shutong Wu, Li Li, and Xuelian Liu. 2024. A High-Linear Radio Frequency Quadrature Modulator with Improved Sideband Suppression and Carrier Leakage Performance. *Tsinghua Science and Technology* 29, 3 (2024), 635–643. <https://doi.org/10.26599/TST.2023.9010024>
- [16] Chenning Li and Zhichao Cao. 2023. LoRa Networking Techniques for Large-scale and Long-term IoT: A Down-to-top Survey. *Comput. Surveys* 55, 3 (2023), 52:1–52:36.
- [17] Chenning Li, Hanqing Guo, Shuai Tong, Xiao Zeng, Zhichao Cao, Mi Zhang, Qiben Yan, Li Xiao, Jiliang Wang, and Yunhao Liu. 2021. NELoRa: Towards Ultra-low SNR LoRa Communication with Neural-enhanced Demodulation. In *Proceedings of ACM SenSys*.
- [18] Chenning Li, Xiuzhen Guo, Longfei Shangguan, Zhichao Cao, and Kyle Jamieson. 2022. CurvingLoRa to Boost LoRa Network Throughput via Concurrent Transmission. In *Proceedings of USENIX NSDI*.
- [19] Chenning Li, Yidong Ren, Shuai Tong, Shakhrol Iman Siam, Mi Zhang, Jiliang Wang, Yunhao Liu, and Zhichao Cao. 2024. ChirpTransformer: Versatile LoRa Encoding for Low-power Wide-area IoT. In *Proceedings of ACM MobiSys*.
- [20] Jansen Christian Liando, Amalinda Gamage, Agustinus W. Tengourtius, and Mo Li. 2019. Known and Unknown Facts of LoRa: Experiences from a Large-scale Measurement Study. *ACM Transactions on Sensor Networks* 15, 2 (2019), 16:1–16:35.
- [21] Daibo Liu, Zhichao Cao, Mengshu Hou, Huigui Rong, and Hongbo Jiang. 2020. Pushing the Limits of Transmission Concurrency for Low Power Wireless Networks. *ACM Transactions on Sensor Networks* 16, 4 (2020), 40:1–40:29.
- [22] Jun Liu, Weitao Xu, Sanjay Jha, and Wen Hu. 2020. Nephelai: towards LPWAN C-RAN with physical layer compression. In *Proceedings of ACM MobiCom*.
- [23] Semtech LoRa. 2024. Semtech SX1276. <https://www.semtech.com/products/wireless-rf/loro-connect/sx1276>. Accessed 11-Aug-2024.
- [24] Qiang Ma, Zhichao Cao, Wei Gong, and Xiaolong Zheng. 2021. BOND: Exploring Hidden Bottleneck Nodes in Large-scale Wireless Sensor Networks. *ACM Transactions on Sensor Networks* 17, 2 (2021), 13:1–13:21.
- [25] Luz E. Marquez, Alfonso Osorio, Maria Calle, Juan Carlos Velez Diaz, Antonio Serrano, and John E. Candelo. 2020. On the Use of LoRaWAN in Smart Cities: A Study With Blocking Interference. *IEEE Internet Things Journal* 7, 4 (2020), 2806–2815.
- [26] Juha Petäjäjärvi, Konstantin Mikhaylov, Antti Roivainen, Tuomo Hänninen, and Marko Pettissalo. 2015. On the coverage of LPWANs: range evaluation and channel attenuation model for LoRa technology. In *Proceedings of ITS Telecommunications*.
- [27] Yidong Ren, Puyu Cai, Jinyan Jiang, Jialuo Du, and Zhichao Cao. 2023. Prism: High-throughput LoRa Backscatter with Non-linear Chirps. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*.
- [28] Yidong Ren, Li Liu, Chenning Li, Zhichao Cao, and Shigang Chen. 2022. Is LoRaWAN Really Wide? Fine-grained LoRa Link-level Measurement in An Urban Environment. In *Proceedings of IEEE ICNP*.
- [29] Yidong Ren, Wei Sun, Jialuo Du, Huaili Zeng, Younsuk Dong, Mi Zhang, Shigang Chen, Yunhao Liu, Tianxing Li, and Zhichao Cao. 2024. Demeter: Reliable Cross-soil LPWAN with Low-cost Signal Polarization Alignment. In *Proceedings of ACM MobiCom*.
- [30] Ettus Research. 2024. SBX. <https://kb.ettus.com/SBX>. Accessed 11-Aug-2024.
- [31] Ettus Research. 2024. USRP N210. <https://www.ettus.com/all-products/un210-kit>. Accessed 11-Aug-2024.
- [32] T. Sakamoto, T. Kawanishi, and M. Izutsu. 2006. Continuous-phase frequency-shift keying with external modulation. *IEEE Journal of Selected Topics in Quantum Electronics* 12, 4 (2006), 589–595.
- [33] Vincent Savaux. 2022. A low-complexity demodulation for oversampled LoRa signal. *International Journal of Mobile Network Design and Innovation* 10, 3 (2022), 148–153.
- [34] Jianxu Sun and Yujian Cheng. 2024. Blindness-free beam scanning antenna with array of array architecture: principle, design, and experiment. *Science China Information Sciences* 67, 3 (2024), 132301.
- [35] Xianjin Xia, Qianwu Chen, Ningning Hou, Yuanqing Zheng, and Mo Li. 2023. XCopy: Boosting Weak Links for Reliable LoRa Communication. In *Proceedings of ACM MobiCom*.

- [36] Zhenqiang Xu, Shuai Tong, Pengjin Xie, and Jiliang Wang. 2022. From Demodulation to Decoding: Toward Complete LoRa PHY Understanding and Implementation. *ACM Transactions on Sensor Networks* 18, 4 (2022), 64:1–64:27.
- [37] Xinghua Yang, Zheyu Liu, Kechao Tang, Xunzhao Yin, Cheng Zhuo, Qi Wei, and Fei Qiao. 2023. Breaking the energy-efficiency barriers for smart sensing applications with “Sensing with Computing” architectures. *Science China Information Sciences* 66, 10 (2023), 200409.